birdclef-2026

6th Place Solution

Rank	6
Team	Sinan Calisir
Author	Sinan Calisir
Topic ID	704949
Version	Original

Hi everyone,

Huge thanks to the organizers and the host. And congratulations to all winners. I truly enjoyed this competition over the last two months.

> **TL;DR:** Soft pseudo-labels generated independently by diverse backbones, a shared-spectrogram OpenVINO ensemble, and rank-blended with the public Perch model.

Introduction

I've competed in previous BirdCLEFs and several other audio competitions, and the winning theme was always the same: diversity, pretraining, pseudo-labelling with more data, and smart post-processing tricks. I followed that recipe and tried to diversify every stage of my solution.

My solution (like several other teams') builds on last year's 1st & 2nd place solutions. I used the 2nd place code as my starting point. Huge thanks to @vialactea & @vladimirsydor for making it available here.

Validation

Soundscape-only validation: file-level splits stratified by site, with **all** train_audio used for training in every fold. Beyond that, I only accepted moves that also looked reasonable on the LB. A bit loose, but previous years showed similar signals, so I went with it.

Modelling

Common setup across all backbones, 5-second windows:

- **Mel:** sample_rate=32000, n_mels=128, n_fft=2048, hop_length=512, f_min=20, normalized=True, fixed_amplitude_to_db=True
- **Augmentation:** mixup p=0.5 (waveform-level), SpecAugment (freq/time masks, p=0.3), RandomFiltering(min_db=-20)
- **Loss / sampling:** BCE focal loss, class-balanced sampler with sqrt class weights
- **Optimization:** cosine decay to 1e-6, label smoothing 0.05, batch size 64 (SwinV2: 32), SWA over the top-3 checkpoints by val ROC-AUC
- **Per-backbone:** RAdam lr=1e-3 + fp32 for ECA-NFNet & EfficientNetV2; AdamW lr=5e-4, wd=0.05 + bf16-mixed for SwinV2
- **Epochs:** 30–50, fewer for later pseudo-label rounds; the final models trained for only 10

SwinV2 requires square inputs, so its branch resizes the mel to 256×256.

Pseudolabelling

The loop:

- train on labelled soundscapes + train_audio
- relabel the unlabelled soundscapes

- retrain
- repeat until no further improvement.

3 rounds for ECA-NFNet and SwinV2, 2 for EfficientNetV2 (started from last year's `tf_efficientnetv2_s_in21k_pretrain_from_bigXCV2Ext_swa.ckpt`). Each track ran independently, and I ensembled the final labels for the final trainings.

Without pseudo-labels the models scored 0.900–0.915; with the final labels, 5-fold ECA-NFNet and SwinV2 reached 0.945–0.950 on their own.

- Labels were **soft**: keep a segment if any species hits p $\geq$ 0.55, zero probabilities below 0.05, and train directly on the raw teacher probabilities. The 0.55 threshold was sanity-checked on held-out S22 segments (lowest threshold with precision $\geq$ 0.8), set slightly lower than the hard-label optimum since soft targets carry their own uncertainty. I also experimented with various thresholds, but 0.55 was consistently working better.
- For extra diversity, the EfficientNet teacher's probabilities got temporal smoothing (averaging with neighbouring segments) and a power transform (1.5) before thresholding.

Ensemble

All models share the same mel input, so I exported each ensemble as a **single OpenVINO graph**: raw waveform in `(batch, 160000)`, log-mel computed **once**, each sub-model branches off it (e.g. SwinV2's 256×256 resize), and the graph outputs the internally weighted-average probabilities `(batch, 234)`. This approach saved me 7-8 minutes for the ensemble models.

Final ensembles from my models:

- ECA-NFNet + EfficientNetV2 - 5 folds each
- ECA-NFNet + SwinV2 - 2 folds each (due to time limits)

Each ran in ~40–45 min and scored 0.950–0.951 public / 0.954 private.

Public Perch/ProtoSSM-Distilled SED branch + final blend

Seeing how strong Perch was in the public notebooks, and having none in my ensemble, adding it was the natural next step. I spent two or three weeks trying to build my own Perch pipeline, but couldn't make it competitive, so I integrated the public notebook directly. Not much contribution from my side here; I mainly made sure the trained models were loaded instead of retrained on every run, to save inference time. I rank-blended it with my ensemble at equal weights:

```
out[prob_cols] = 0.5 * a[prob_cols].rank(pct=True).values + 0.5 * b[prob_cols].rank(pct=True).values
```

TTA also improved results consistently, but it competes with ensemble size for inference time so I chose to spend the budget on more models. Both selected submissions reached gold; the better one (ECA-NFNet + SwinV2, blended with Perch) finished at **0.95762 private**.

Things that didn't work

- 10/20-second inputs.
- Post-processing beyond temporal smoothing for my ensemble (top-k, file-level scaling). Only temporal smoothing showed consistent improvements. I think this is because of how I handle the pseudo-labels with thresholds in my pipeline.
- Other backbones (`resnest50d, convnext, seresnext26d, regnety_040, rexnet_150`) didn't show significant improvement, so to save compute I didn't carry them through the multi-round setup.
- Hard labels instead of the soft ones.
- Perch with my pseudolabelling recipe that I applied to the other models.

Some working notes with the coding tools

I used Claude Code throughout the competition, mainly for adapting the original code base. A few observations:

- **Ideas came from me, implementation from Claude.** Whenever I expected creativity for new experiments, it fell short. I decided the next steps and let it handle the code.
- **Memory was a double-edged sword.** Claude saved the "important" parts of experiment discussions to its memory, but sometimes that included dead experiments. Days later it would compare new runs against ideas we had already discarded. Managing this with a growing experiment list became a burden.
- **Context advantage.** I integrated W&B so the agent could see results, but with a loose validation set, my knowledge from the public LB, the W&B logs, and past solutions was a significant advantage over the model's own context.

Thank you for reading!

References

- https://www.kaggle.com/competitions/birdclef-2025/writeups/nikita-babych-1st-place-solution-multi-iterative-n
- https://www.kaggle.com/competitions/birdclef-2025/writeups/volodymyr-vialactea-2nd-place-journey-down-the-rab
- https://github.com/VSydorskyy/BirdCLEF_2025_2nd_place

- https://www.kaggle.com/code/tuckerarrants/bc2026-distilled-sed